TP1 — Mise en place du projet TaskFlow

Module 1 · Premier écran de l'application fil rouge

Élément	Détail
Projet fil rouge	TaskFlow — application mobile de gestion de tâches, construite de TP en TP
Point de départ	Aucun prérequis de code : vous partez d'un dossier vide.
Point d'arrivée	Un écran d'accueil statique affichant 5 tâches avec TacheCard.
Git	git init dès la création du projet. À la fin : commit, puis git tag tp1-fin.

Où se situe ce TP dans le projet

TP	Étape	Ce que l'application gagne
TP1 (ce TP)	Mise en place	Projet Expo, structure src/, composant TacheCard, écran d'accueil statique
TP2	Interactif & navigable	State des tâches (ajout, bascule, suppression), FlatList, navigation Stack + Tab
TP3	Base de données	Base MySQL taskflow : 3 tables, données de test, requêtes CRUD, JOIN, agrégations
TP4	API & connexion	API Express CRUD, service axios, écrans branchés sur l'API (loading / error / success)
TP5	Application connectée	Lieu (GPS), photo (caméra), rappels (notifications), secousse (capteurs), synchronisation temps réel (WebSocket)
TP6	Sécurité & finalisation	bcrypt, JWT, SecureStore, navigation conditionnelle,

© 2026 Caplogy. All rights reserved.

		finitions, soutenance
TP7	Optimisation (bonus)	Mémoïsation, pagination, recherche avec debounce, index SQL, mesures

0. Le projet fil rouge : TaskFlow

Tout au long de ce cours, vous allez construire progressivement une application mobile complète appelée TaskFlow : une application de gestion de tâches collaborative. Chaque module ajoute une brique à ce même projet — vous ne recommencez jamais de zéro.

Module	Ce que TaskFlow gagne à cette étape
Module 1 (ce TP)	Projet créé, écran d'accueil statique listant des tâches
Module 2	Ajout/suppression/validation de tâches, navigation entre écrans
Module 3	Base de données MySQL : utilisateurs, tâches, catégories
Module 4	API REST (Node/Express) + connexion réelle depuis React Native
Module 5	Fonctionnalités connectées : géolocalisation, photo, notifications, capteurs, temps réel
Module 6	Authentification sécurisée, finitions, application complète et soutenable
Module 7 (bonus)	Optimisation et performance

> ⚠️ **Attention**
>
> Conservez précieusement le dossier de votre projet et versionnez-le avec Git après chaque TP : vous en aurez besoin jusqu'au dernier module.

1. Objectifs pédagogiques du TP1

© 2026 Caplogy. All rights reserved.

- Installer et vérifier un environnement de développement React Native complet.
- Créer un projet Expo fonctionnel depuis la ligne de commande.
- Organiser un projet selon une structure de dossiers professionnelle.
- Créer un composant réutilisable (TacheCard) en utilisant les props.
- Construire un écran d'accueil statique avec Flexbox et StyleSheet.

2. Prérequis

Critère	Validé
Node.js (version LTS) installé et vérifié (node -v, npm -v)	☐
Visual Studio Code installé avec les extensions recommandées	☐
Application Expo Go installée sur un smartphone	☐
Compte Expo créé (facultatif mais recommandé)	☐
Un dossier de travail dédié au cours créé sur l'ordinateur	☐

3. Partie 1 — Créer le projet Expo

3.1 Génération du projet

Ouvrez un terminal dans votre dossier de travail et exécutez les commandes suivantes :

terminal

```
npx create-expo-app TaskFlow
cd TaskFlow
npx expo start
```

Scannez le QR code affiché avec l'application Expo Go pour vérifier que le projet par défaut s'affiche correctement sur votre téléphone.

3.2 Vérification

© 2026 Caplogy. All rights reserved.

- L'application affiche l'écran par défaut d'Expo ("Open up App.js to start working!").
- Aucune erreur rouge n'apparaît dans le terminal ou sur le téléphone.

4. Partie 2 — Organiser la structure du projet

Créez la structure de dossiers suivante à l'intérieur de TaskFlow, en vous appuyant sur le cours :

arborescence

```
TaskFlow/
├── App.js
├── src/
│   ├── components/
│   │   └── TacheCard.js
│   ├── screens/
│   │   └── AccueilScreen.js
│   └── styles/
│       └── couleurs.js
└── assets/
```

Dans src/styles/couleurs.js, définissez un petit fichier de constantes de couleurs que vous réutiliserez dans toute l'application :

src/styles/couleurs.js

```
export const COULEURS = {
  fond: '#F5F5F5',
  texte: '#222222',
  primaire: '#44445A',
  accent: '#71D2E3',
  muted: '#8B8BA3',
};
```

5. Partie 3 — Créer le composant TacheCard

Ce composant réutilisable affichera une tâche unique (titre, catégorie, statut). Il doit recevoir ses données via des props, jamais en dur.

5.1 Ce qu'on attend du composant

- Un titre de tâche (texte en gras).
- Une catégorie (petit badge de couleur).
- Un indicateur visuel si la tâche est terminée ou non (ex : icône ou couleur différente).

© 2026 Caplogy. All rights reserved.

- Le composant doit être entièrement piloté par ses props : titre, categorie, terminee.

> 💡 **Indice**
>
> Pensez à utiliser un objet de styles avec StyleSheet.create, et à conditionner une couleur ou un texte selon la prop terminee (ex : via un opérateur ternaire).

6. Partie 4 — Construire l'écran d'accueil (AccueilScreen)

Dans src/screens/AccueilScreen.js, créez l'écran d'accueil de TaskFlow :

- Définir un tableau JavaScript de tâches fictives (au moins 5), chacune avec un id, un titre, une categorie et un booléen terminee.
- Afficher un en-tête avec le titre de l'application ("TaskFlow") et un sous-titre.
- Parcourir le tableau de tâches avec .map() pour afficher un TacheCard par tâche.
- Englober la liste dans un ScrollView pour permettre le défilement si la liste est longue.
- Mettre à jour App.js pour qu'il affiche AccueilScreen comme composant racine.

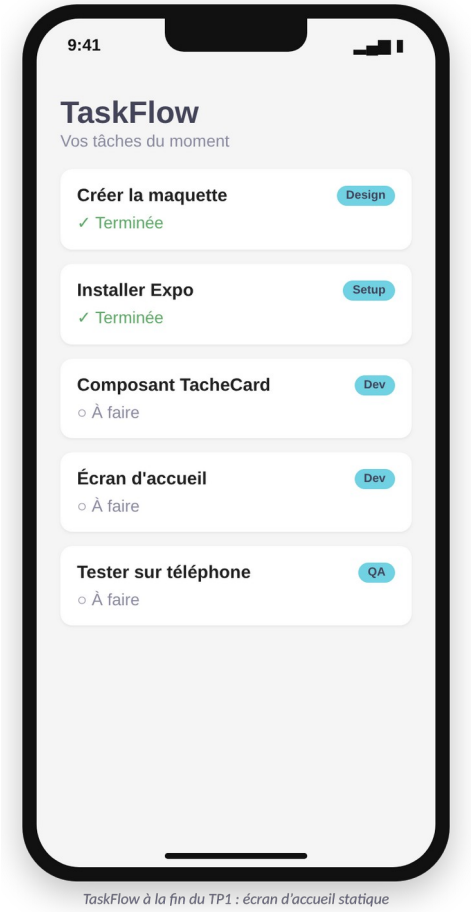
7. Partie 5 — Mise en forme (Flexbox & StyleSheet)

- Utilisez flexDirection, justifyContent et gap pour espacer proprement les cartes de tâches.
- Donnez à l'écran un fond cohérent avec la palette définie dans couleurs.js.
- Assurez-vous qu'aucun texte ne déborde et que les marges sont cohérentes sur tout l'écran.
- Testez le rendu sur votre téléphone via Expo Go après chaque modification importante.

Rendu attendu

Les captures ci-dessous montrent le résultat à obtenir. Les écrans de téléphone sont des maquettes fidèles aux styles du cours ; les captures de terminal proviennent d'exécutions réelles.

© 2026 Caplogy. All rights reserved.



TaskFlow à la fin du TP1 : écran d'accueil statique

Rendu attendu de l'écran d'accueil à la fin du TP1

```
Terminal — arborescence attendue du TP1

etudiant@mac ~/TaskFlow $ find . -not -path "./.git*" | sort
.
./App.js
./src
./src/components
./src/components/TacheCard.js
./src/screens
./src/screens/AccueilScreen.js
./src/styles
./src/styles/couleurs.js
```

Arborescence attendue du projet (capture réelle)

8. Livrables attendus

Critère	Validé

© 2026 Caplogy. All rights reserved.

Un projet Expo nommé TaskFlow, fonctionnel, lancé sans erreur	☐
La structure de dossiers src/components, src/screens, src/styles respectée	☐
Un composant TacheCard réutilisable piloté par ses props	☐
Un écran AccueilScreen affichant au moins 5 tâches fictives	☐
Une mise en forme soignée avec Flexbox et StyleSheet	☐
Un premier commit Git du projet (git init, git add ., git commit)	☐

9. Critères d'évaluation

Critère	Pondération
Projet fonctionnel (lancement sans erreur)	20%
Structure de dossiers respectée	15%
Qualité et réutilisabilité du composant TacheCard	25%
Qualité de la mise en page (Flexbox, StyleSheet)	25%
Propreté du code (nommage, organisation)	15%

i Information

Pour aller plus loin
Une fois le TP terminé, comparez votre travail à la correction fournie. N'hésitez pas à améliorer visuellement votre écran d'accueil avant de passer au Module 2, qui rendra cette liste interactive.

Avant de passer au TP suivant

- Exécuter l’application de bout en bout et vérifier chaque point de la grille.

© 2026 Caplogy. All rights reserved.

- git add . && git commit -m "TP1 terminé" && git tag tp1-fin
- Le TP2 repartira exactement de ce tag : ne supprimez rien.

© 2026 Caplogy. All rights reserved.